DFSha

Sistema de archivos distribuido con alta disponibilidad, rendimiento y seguridad

Informe técnico · Hito 4 — Entrega final

Curso	ST0263 Tópicos Especiales en Telemática / SI3007 Sistemas Distribuidos
Periodo	2026-2 · Universidad EAFIT
Integrantes	Andrés Restrepo Giraldo Valentina Zapata Acosta Sebastián Granda Giraldo
Arquitectura	Opción 1 — Cliente/Servidor, DFS por bloques estilo HDFS
Repositorio	github.com/AndyRestrePO/DFSha
Fecha	Octubre de 2026

En pocas palabras. DFSha funciona como una biblioteca con un bibliotecario y varias bodegas. Cada archivo se corta en pedazos de 4 MB; el **ControlNode** (el bibliotecario) solo lleva el catálogo de qué pedazo está en qué bodega, y los **DataNodes** (las bodegas) guardan los pedazos, cada uno en dos bodegas distintas y bajo llave. Si una bodega se cierra, el bibliotecario lo nota en segundos, sigue entregando la copia de la otra bodega y saca una copia nueva para volver a tener dos.

1. Objetivo

Diseñar e implementar un sistema de archivos distribuido (DFS) que permita almacenar y recuperar archivos grandes repartidos entre varios nodos, de forma que tanto la escritura como la lectura sean distribuidas, y que cumpla requisitos de escalabilidad, alta disponibilidad, consistencia, rendimiento, seguridad y transparencia.

Objetivos específicos

- Separar el plano de control (metadatos) del plano de datos (bloques), al estilo de GFS y HDFS.
- Particionar cada archivo en bloques y replicarlo en al menos dos DataNodes.
- Detectar fallas de nodos y recuperarse sin intervención manual (re-replicación automática).
- Garantizar confidencialidad en tránsito (TLS) y en reposo (AES-256-GCM), con control de acceso por usuario, 2FA y API keys.
- Ofrecer al usuario un cliente con abstracción jerárquica tipo Linux y una API REST documentada.

2. Marco teórico breve

Un sistema de archivos distribuido permite compartir y acceder de forma concurrente a archivos almacenados en varios nodos, ocultando al usuario dónde están físicamente. Sistemas clásicos como NFS, AFS o SMB/CIFS se integran con el sistema operativo y ofrecen transparencia total; los almacenamientos de objetos como AWS S3 exponen en cambio una API o SDK y tratan el archivo como un todo, con un modelo de escritura única y muchas lecturas (WORM).

Google File System (GFS) y su versión abierta **HDFS** popularizaron el diseño maestro–trabajadores: un nodo maestro (Master / NameNode) guarda solo metadatos y muchos servidores de datos (ChunkServers / DataNodes) guardan bloques grandes (64–128 MB) replicados, normalmente tres veces. Los clientes piden al maestro la ubicación de los bloques y luego transfieren los datos directamente con los servidores de datos, lo que evita que el maestro sea un cuello de botella. El maestro detecta fallas por heartbeats y ordena re-replicar los bloques que pierden copias.

DFSha adopta este modelo por bloques, con bloques de 4 MB (adecuados para las pruebas en máquinas pequeñas de AWS Academy) y factor de replicación configurable (2 por defecto).

3. Descripción del servicio y problema abordado

El problema es guardar archivos que pueden ser más grandes que el disco de una sola máquina y seguir sirviéndolos aunque algún servidor falle, sin que el usuario tenga que saber en qué máquina está cada parte. DFSha resuelve esto con un servicio C/S autónomo (Opción 1): el servicio corre dentro de una red privada (VPC de AWS o red de Docker) y expone una API abierta hacia los clientes.

Requisitos y cómo se cumplen

Req.	Descripción	Implementación en DFSha
RF1	Gestión del FS: ls, cd, mkdir, rmdir, rm	Endpoints /fs/ls, /fs/mkdir, /fs/rmdir, /fs/rm; *cd* y rutas relativas en el shell del cliente.
RF2	Transferencia: send/receive	*put*: /fs/create → PUT de bloques a los DataNodes → /fs/commit. *get*: /fs/locate → GET de cada bloque, probando réplicas alternas si una falla.
RF3	Acceso: open, close, read, write, lock	Handles con modos r, r+ y w; lectura por rango (offset/length) y escritura en offset con lock exclusivo.
RNF1	Escalabilidad	DataNodes se agregan en caliente (se registran solos); el ControlNode no transporta datos en put/get.
RNF2	Alta disponibilidad	Réplicas en nodos distintos, heartbeat, lectura con failover, re-replicación automática, restart automático de contenedores, estado persistente.
RNF3	Consistencia	Escritura en dos fases (create → commit verificado), locks exclusivo/compartido, archivos no confirmados invisibles.
RNF4	Particionamiento	Bloques de 4 MB repartidos por round-robin entre los DataNodes activos.
RNF5	Rendimiento / concurrencia	Transferencia directa cliente↔DataNode, servidores ASGI concurrentes, locks por archivo (no globales) para el I/O.
RNF6	Seguridad	TLS en todos los enlaces, AES-256-GCM en disco, PBKDF2 para contraseñas, ACL por dueño, roles, TOTP 2FA, API keys, tokens HMAC por bloque, secreto interno entre nodos.
RNF7	Transparencia	Ubicación dinámica: el cliente nunca conoce de antemano los DataNodes, los recibe del ControlNode en cada operación.

4. Arquitectura del sistema

Tipo: Maestro–Trabajadores (Master–Workers) sobre un esquema Cliente/Servidor. El servidor está compuesto por un ControlNode y N DataNodes que se comunican entre sí (S2S) dentro de una red privada; el cliente solo conoce la URL del ControlNode.

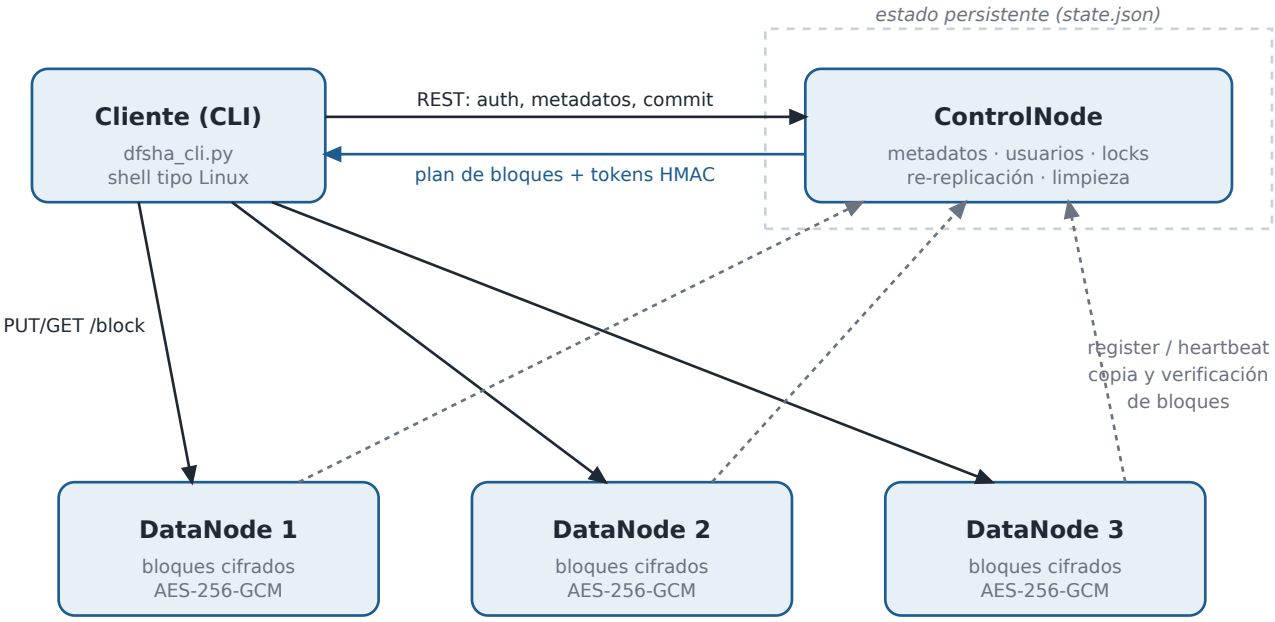


Figura 1. Componentes de DFSha y sus canales de comunicación.

Componente	Responsabilidad	Estado que guarda
ControlNode	Autenticación y autorización; árbol de directorios; tabla archivo → bloques → réplicas; plan de ubicación; sesiones y locks; monitoreo de DataNodes; re-replicación y limpieza de huérfanos.	state.json (directorios, dueños, archivos, usuarios, API keys) escrito de forma atómica; DataNodes y sesiones en memoria (se reconstruyen solos).
DataNode	Guardar, entregar y borrar bloques; validar el token HMAC de cada bloque; cifrar/descifrar; reportar heartbeat.	Un archivo por bloque en /app/blocks (volumen Docker).
Cliente	CLI y shell; cortar archivos en bloques; subir cada réplica; reconstruir el archivo y probar réplicas alternas si una falla.	Ninguno (sin estado).

Flujo de escritura (put)

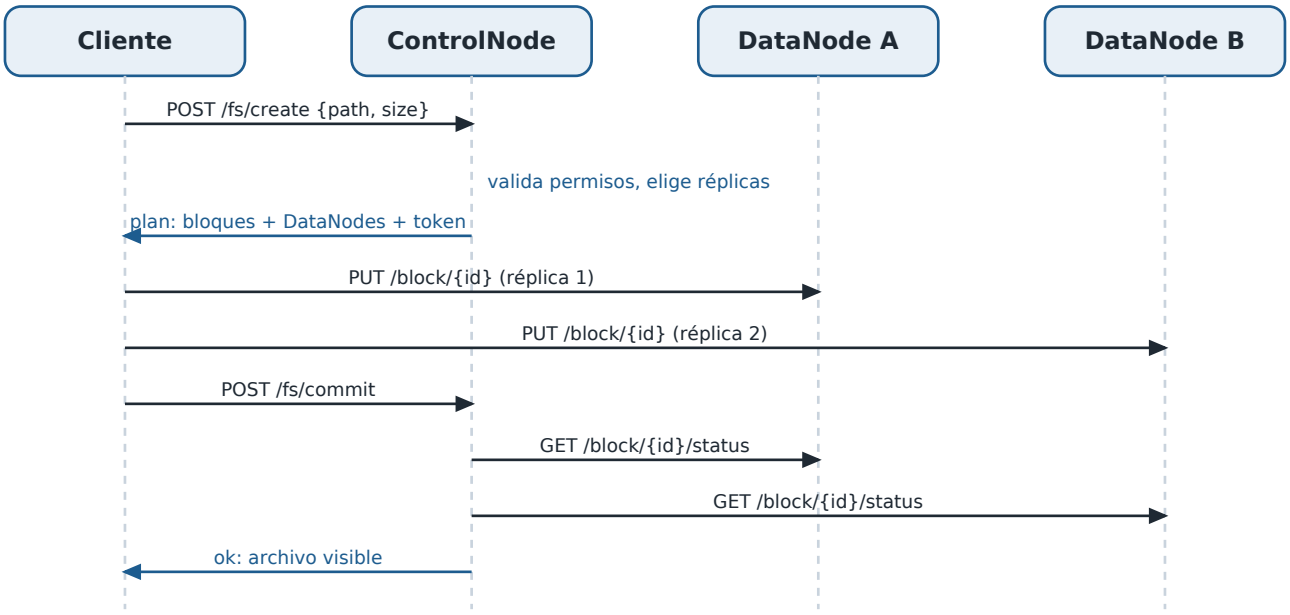


Figura 2. Subida de un archivo. Si cualquier paso falla, el cliente llama /fs/abort y se borran los bloques parciales.

Flujo de falla y recuperación

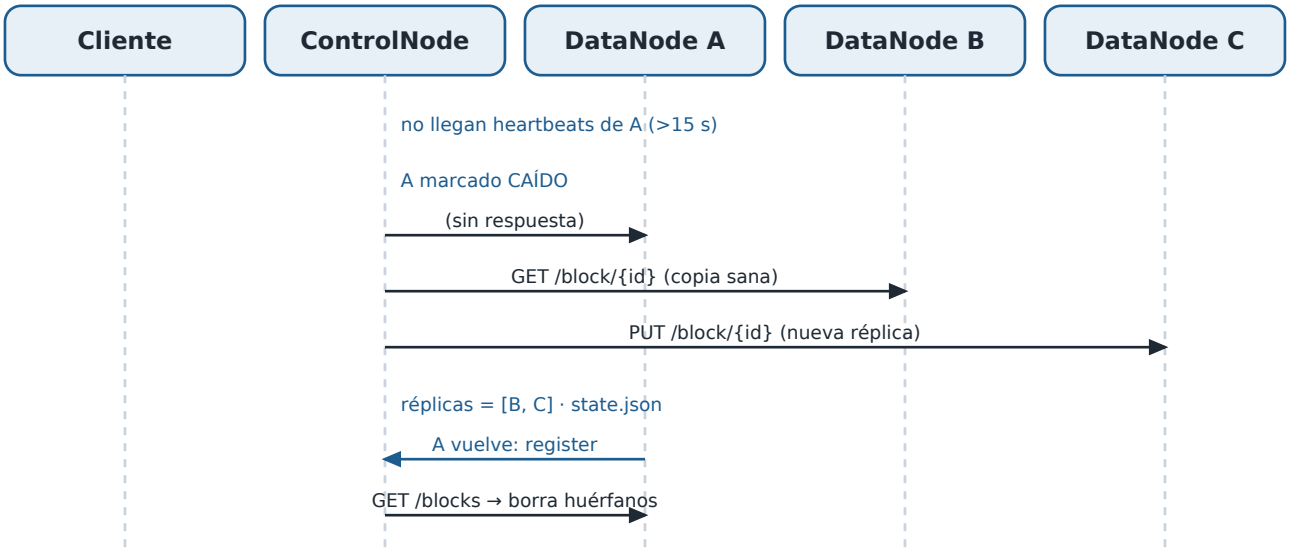


Figura 3. Re-replicación automática y limpieza cuando un DataNode cae y luego regresa.

5. Especificación de protocolos y APIs

Toda la comunicación usa **REST sobre HTTP/1.1**, cifrada con **TLS** cuando se activa el modo seguro. Se eligió REST por ser fácil de depurar, de documentar (FastAPI genera OpenAPI en */docs*) y de atravesar firewalls; los bloques viajan como cuerpo binario (*application/octet-stream*), no en JSON, para no pagar el costo de Base64 en las transferencias grandes.

Canales de comunicación

Canal	Uso	Autenticación
Cliente ↔ ControlNode	Metadatos: login, ls, mkdir, create, locate, commit, open/read/write/lock, admin.	HTTP Basic + TOTP (X-TOTP-Code) o X-API-Key.
Cliente ↔ DataNode	Datos: PUT/GET de bloques.	X-Block-Token = HMAC-SHA256(secreto, block_id) entregado por el ControlNode.
ControlNode ↔ DataNode	Registro y heartbeat (DN→CN); verificación en commit, lectura/escritura para read/write y re-replicación, borrado, listado de bloques (CN→DN).	X-Internal-Secret (registro, heartbeat, listado) y X-Block-Token.
DataNode ↔ DataNode	No hay comunicación directa: la copia entre nodos la coordina el ControlNode, lo que simplifica la seguridad.	—
ControlNode ↔ ControlNode	No aplica: hay un único ControlNode (ver limitaciones).	—

API del ControlNode

Método y ruta	Entrada	Salida / efecto
POST /auth/login	credenciales	{status, user}
POST /auth/users · DELETE /auth/users/{u}	{username, password}	crea / borra usuario (solo admin)
POST /auth/totp/setup · /confirm · DELETE /auth/totp	{code}	secreto y URI otpauth; activa o desactiva 2FA
POST /auth/api-keys · DELETE /auth/api-keys/{id}	{name}	{key_id, api_key} (se guarda solo el hash)
GET /fs/ls?path=	ruta	{directories, files[{name,size}]}
POST /fs/mkdir · DELETE /fs/rmdir · DELETE /fs/rm	{path}	crea / borra
POST /fs/create	{path, size}	{block_size, blocks[{block_id, seq, replicas[], block_token}]}
POST /fs/commit · DELETE /fs/abort	{path}	confirma verificando cada réplica / cancela
GET /fs/locate?path=	ruta	{size, blocks[...]} con réplicas activas primero
POST /fs/open	{path, mode: r | r+ | w}	{handle, size}
POST /fs/read · /fs/write	{handle, offset, length | data(base64)}	datos leídos / tamaño nuevo
POST /fs/lock · /fs/unlock · /fs/close	{handle}	toma o libera el lock exclusivo, cierra la sesión
GET /admin/cluster	—	nodos, bloques por nodo, sub-replicados, estadísticas de mantenimiento
POST /datanode/register · /heartbeat	{datanode_id, url, internal_url}	registro / latido (interno)

API del DataNode

Método y ruta	Descripción
PUT /block/{id}	Guarda el bloque (cuerpo binario), cifrado y con escritura atómica (archivo .tmp + rename).
GET /block/{id}	Devuelve el bloque descifrado; si la etiqueta GCM no coincide (bloque alterado) responde error.
DELETE /block/{id}	Borra el bloque.
GET /block/{id}/status	{exists, size}; lo usa el commit para verificar réplicas.
GET /blocks	Lista de bloques y su edad; solo para el ControlNode (limpieza de huérfanos).
GET /health	Estado y número de bloques.

Ejemplo de plan devuelto por /fs/create:

```
{"path": "/docs/video.mp4", "block_size": 4194304,
 "blocks": [{"block_id": "6294f35a-...", "seq": 0,
   "replicas": [{"datanode_id": "dn1", "datanode_url": "https://54.x.x.x:8001"},
                {"datanode_id": "dn2", "datanode_url": "https://3.x.x.x:8002"}],
   "block_token": "9c1e...a7"}, ...]}
```

6. Algoritmos de particionamiento, distribución y consistencia

6.1 Particionamiento

Un archivo de tamaño S se divide en ⌈S / 4 MB⌉ bloques (mínimo uno, para los archivos vacíos). Cada bloque recibe un identificador UUID4 aleatorio, que además sirve como nombre de archivo en el DataNode y como dato asociado (AAD) en el cifrado, de modo que un bloque no puede ser movido o renombrado sin que se detecte.

6.2 Ubicación de réplicas (round-robin con desplazamiento)

Con la lista ordenada de DataNodes activos $A = [a_0, \ldots, a_{n-1}]$ y un contador global c, el bloque k va a los nodos $a_{(c+j) \bmod n}$ para $j = 0 \ldots R-1$, donde R es el factor de replicación. Así las R réplicas siempre quedan en nodos distintos y la carga se reparte de forma pareja. Si hay menos de R nodos activos la subida se rechaza (503) en vez de guardar datos con menos protección de la prometida.

```
activos = ordenar(datanodes con heartbeat reciente)
si len(activos) < R: error 503
para cada bloque k:
    inicio = contador++ mod len(activos)
    replicas[k] = [activos[(inicio + j) mod len(activos)] para j en 0..R-1]
```

6.3 Detección de fallas y re-replicación

Cada DataNode envía un heartbeat cada 5 s; si el ControlNode no recibe uno en HEARTBEAT_TIMEOUT (15 s) el nodo se considera caído. Un hilo de mantenimiento corre cada REPAIR_INTERVAL (10 s):

```
para cada bloque confirmado:
    vivas = réplicas que están activas
    si vivas == 0: contar como perdido
    si 0 < len(vivas) < R:
        destinos = nodos activos sin el bloque, ordenados por menos bloques
        datos = GET del bloque desde una réplica viva
        PUT de datos en destinos[0 .. R-len(vivas)-1]
        con el lock: si el bloque sigue en la metadata -> réplicas = vivas + nuevas
                     si no (el archivo cambió)        -> borrar las copias nuevas
```

Elegir los destinos por menor número de bloques evita que la recuperación concentre todo en un solo nodo. Cuando un nodo caído vuelve, sus copias viejas ya no aparecen en la metadata; la segunda tarea del hilo lista los bloques de cada DataNode y borra los que no están referenciados para ese nodo y tienen más de ORPHAN_GRACE segundos (para no tocar bloques de una subida en curso). La metadata se consulta *después* de listar el disco, para que un bloque recién creado siempre aparezca referenciado.

6.4 Consistencia

- **Escritura en dos fases.** /fs/create registra el archivo como *no listo*; solo /fs/commit, tras verificar que cada réplica existe y tiene el tamaño esperado, lo hace visible. Un lector nunca ve un archivo a medias.
- **Réplicas idénticas.** Un bloque nunca se modifica en sitio: *write* genera bloques nuevos y luego cambia la metadata de forma atómica, así todas las réplicas de un bloque tienen siempre el mismo contenido (consistencia fuerte por inmutabilidad, como en los almacenamientos de objetos).
- **Locks.** Lock exclusivo por archivo para escribir; no se otorga si hay lectores abiertos y mientras exista no se pueden abrir lectores (modelo lectores–escritor). Las sesiones vencen tras HANDLE_TIMEOUT (300 s) de inactividad para que un cliente caído no deje el archivo bloqueado.
- **Metadata durable.** Cada cambio se escribe en state.json mediante archivo temporal + rename atómico, de modo que un corte de energía no deja el estado corrupto.

7. Seguridad

Amenaza	Mecanismo
Espionaje de la red	TLS en todos los enlaces (cliente–ControlNode, cliente–DataNode, ControlNode–DataNode) con una CA propia generada por scripts/gen_certs.sh; los nodos verifican el certificado del otro extremo.
Robo de un disco o volumen	Bloques cifrados con AES-256-GCM (nonce de 96 bits por escritura, block_id como AAD). Secretos TOTP cifrados con Fernet. Contraseñas con PBKDF2-SHA256 (310 000 iteraciones, sal aleatoria). API keys guardadas como hash.
Acceso de un usuario a datos ajenos	ACL por dueño en archivos y directorios; rol admin; ls solo muestra lo propio.
Contraseña filtrada	2FA TOTP opcional por usuario; API keys revocables.
Cliente que pide bloques arbitrarios a un DataNode	Cada operación de bloque exige un token HMAC-SHA256 que solo el ControlNode puede generar y que solo entrega a quien tiene permiso sobre el archivo.
DataNode falso que se registra	Registro y heartbeat exigen el secreto interno compartido (comparación en tiempo constante).

8. Entorno de ejecución

Cada componente tiene su **Dockerfile** (imagen python:3.11-slim con FastAPI y Uvicorn). El mismo contenedor arranca en HTTPS si recibe las variables TLS_CERT y TLS_KEY, o en HTTP si no. Para una sola máquina se usa **docker-compose.yml** (1 ControlNode + 3 DataNodes, volúmenes persistentes y *restart: unless-stopped*) y **docker-compose.tls.yml** para activar TLS.

En **AWS Academy** el despliegue usa una instancia EC2 por nodo dentro del mismo VPC: el ControlNode publica el puerto 8000 y cada DataNode su propio puerto. Cada DataNode anuncia dos direcciones: *DATANODE_URL* (pública, la que reciben los clientes) e *DATANODE_INTERNAL_URL* (privada, la que usa el ControlNode), de modo que el tráfico

interno no sale del VPC. El grupo de seguridad solo abre esos puertos. Los comandos exactos están en el README del repositorio.

Parámetro	Valor
Lenguaje / framework	Python 3.11, FastAPI 0.115, Uvicorn, httpx
Criptografía	cryptography (AES-GCM, Fernet), pyotp, hashlib/hmac
Contenedores	Docker, Docker Compose
Nube	AWS Academy — EC2 Ubuntu, Security Groups, VPC
Configuración por defecto	Bloque 4 MB · R = 2 · heartbeat 5 s · timeout 15 s · reparación cada 10 s

9. Pruebas y análisis de resultados

9.1 Pruebas funcionales automáticas

La carpeta *tests/* contiene una suite con pytest que levanta un clúster real (1 ControlNode + 3 DataNodes, cada uno en su propio proceso y puerto) y ejecuta el cliente tal como lo usaría una persona. La suite pasó completa (15/15) tanto en HTTP como con TLS en todos los enlaces.

Prueba	Qué verifica	Req.	Resultado
put/get archivo de 10 MB	El archivo bajado es idéntico byte a byte; se reparte en 3 bloques	RF2, RNF4	OK
Reparto y réplicas	Cada bloque está en 2 nodos distintos y participan los 3 DataNodes	RNF4, RNF2	OK
mkdir / rmdir / rm	No se borra un directorio con contenido; luego sí	RF1	OK
Cifrado en disco	El bloque en disco empieza con DFSB1 y no contiene el texto original	RNF6	OK
Sin credenciales / clave errada	Respuesta 401	RNF6	OK
Control de acceso	Otro usuario recibe 403 sobre archivos y directorios ajenos; un usuario normal no crea usuarios	RNF6	OK
Tokens e intrusos	DataNode rechaza token falso; registro de nodo sin secreto rechazado	RNF6	OK
2FA y API key	Con 2FA activo el login sin código falla y con código funciona; API key autentica	RNF6	OK
open / lock / write / read	Con lock no se abren lectores; write en offset 3 produce 012ABC6789	RF3, RNF3	OK
Subida duplicada	Segundo put sobre la misma ruta se rechaza	RNF3	OK
Lectura con nodo caído	Se mata dn1 (SIGKILL) y el get sigue devolviendo el archivo íntegro	RNF2	OK
Re-replicación	Sin intervención, todos los bloques vuelven a 2 réplicas activas	RNF2	OK
Escritura tras la falla	Con 2 nodos activos se sigue pudiendo subir	RNF2	OK

Prueba	Qué verifica	Req.	Resultado
Nodo que vuelve	Los bloques viejos del nodo recuperado se borran solos	RNF3	OK
Reinicio del ControlNode	Tras reiniciarlo, la metadata persiste y los DataNodes se re-registran	RNF2	OK

9.2 Rendimiento

Se midió con *tests/benchmark.py* en una máquina de 2 vCPU con los cuatro nodos corriendo localmente, primero en HTTP y luego con TLS. Los tiempos incluyen el arranque del intérprete del cliente (~0,5 s por comando), por eso los archivos pequeños muestran un throughput bajo: domina el costo fijo, no la transferencia.

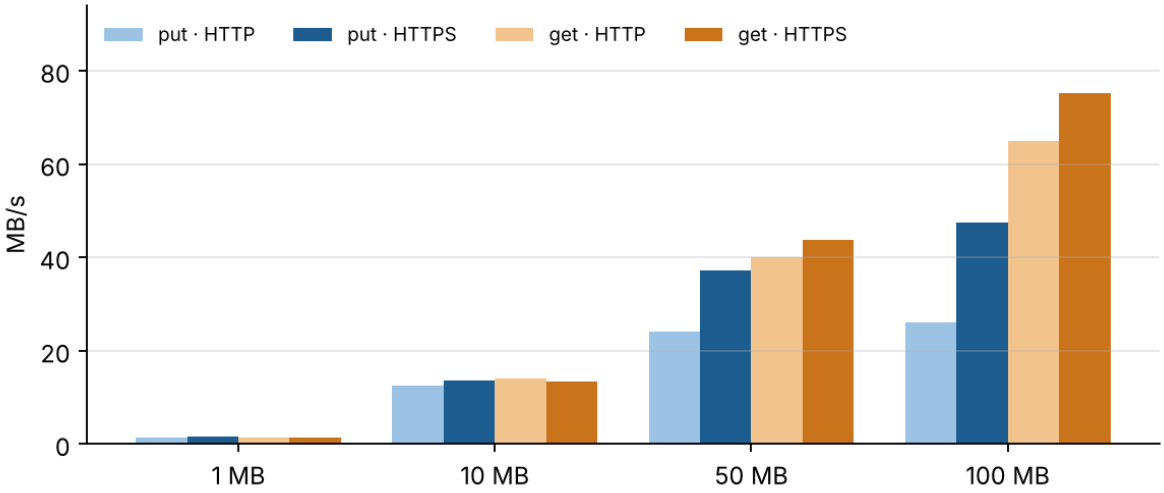


Figura 4. Throughput de subida y bajada según tamaño de archivo (más es mejor).

Tamaño	Bloques	put HTTP	put HTTPS	get HTTP	get HTTPS
1 MB	1	0.72 s	0.58 s	0.67 s	0.69 s
10 MB	3	0.8 s	0.73 s	0.71 s	0.74 s
50 MB	13	2.07 s	1.34 s	1.25 s	1.14 s
100 MB	25	3.84 s	2.11 s	1.54 s	1.33 s

Clientes simultáneos (8 MB c/u)	Tiempo total HTTP	MB/s agregado HTTP	Tiempo total HTTPS	MB/s agregado HTTPS
1	0.71 s	11.2	0.55 s	14.6
4	1.69 s	18.9	1.36 s	23.5
8	3.16 s	20.3	2.7 s	23.7

Recuperación: al matar un DataNode con 45 bloques afectados, el clúster volvió a tener todos los bloques con 2 réplicas en 7.7 s (HTTP) y 6.4 s (HTTPS), usando un timeout de heartbeat de 4 s y reparación cada 2 s en la prueba. Con la configuración por defecto (15 s y 10 s) el tiempo esperado es de unos 25 s.

9.3 Análisis

- El throughput crece con el tamaño del archivo porque el costo fijo (autenticación, plan, arranque del cliente) se reparte entre más bloques; con 100 MB la bajada supera los 60 MB/s.
- La subida es más lenta que la bajada porque el cliente envía cada bloque R = 2 veces (una por réplica) y el commit verifica todas las réplicas; la bajada lee una sola réplica por bloque.
- Con varios clientes simultáneos el throughput agregado sube y luego se aplana: en la prueba los 4 nodos comparten 2 vCPU, así que el límite es la CPU de la máquina. En AWS, con un nodo por instancia, cada DataNode agrega su propio ancho de banda y el ControlNode no participa en la transferencia de datos.
- La diferencia entre HTTP y HTTPS quedó dentro del ruido de la medición: el costo de TLS es pequeño frente al cifrado AES-GCM en disco y al trabajo de Python, lo que justifica dejar TLS activo en producción.
- La recuperación ante la caída de un nodo fue automática y sin pérdida de datos en todas las ejecuciones.

10. Conclusiones, limitaciones y trabajo futuro

DFSha cumple los requisitos funcionales (RF1–RF3) y los no funcionales del enunciado: distribuye y replica los bloques, sigue sirviendo archivos cuando un DataNode cae y se recupera solo, mantiene consistencia mediante escritura en dos fases, bloques inmutables y locks, y protege los datos en tránsito y en reposo. Separar metadatos de datos resultó clave: el ControlNode solo maneja mensajes pequeños y los datos fluyen directamente entre cliente y DataNodes.

Limitaciones y posibles mejoras

- **ControlNode único.** Su estado es persistente y se recupera al reiniciar, pero mientras está caído el servicio no responde. La mejora natural es un ControlNode secundario con la metadata replicada mediante un algoritmo de consenso (Raft) o un log compartido, como el Standby NameNode de HDFS.
- **write completo.** Escribir en un offset reescribe el archivo entero con bloques nuevos; se podría reescribir solo los bloques afectados.
- **Réplicas desde el cliente.** El cliente sube cada réplica; un pipeline DataNode → DataNode reduciría a la mitad el tráfico de subida del cliente.
- **Ubicación consciente de la carga y la red.** Hoy es round-robin; podría considerar espacio libre o zonas de disponibilidad de AWS.

Referencias

- S. Ghemawat, H. Gobioff, S.-T. Leung. *The Google File System*. SOSP 2003.
- K. Shvachko, H. Kuang, S. Radia, R. Chansler. *The Hadoop Distributed File System*. IEEE MSST 2010.
- Enunciado Proyecto 1 DFSha, ST0263/SI3007, Universidad EAFIT, 2026-2.
- Documentación de FastAPI, httpx y cryptography (AES-GCM, Fernet).